



UNIVERSIDADE FEDERAL DO RIO GRANDE DO NORTE

COMPUTAÇÃO DE ALTO DESEMPENHO

DOCENTE: CARLA DOS SANTOS SANTANA

E SAMUEL XAVIER DE SOUZA

DISCENTE: CLARA VIRGÍNIA NASCIMENTO SANTOS (20240002212)

Tarefa 19

1. ENUNCIADO

Faça os exercícios de adição de vetores, `vadd.c`, dos slides 27 e 48 do tutorial de programação de GPUs com OpenMP em um dos nós com GPU do NPAD. Compare os tempos de execução apenas na CPU e com o uso da GPU. Reporte seu progresso, apresentando os problemas encontrados e as soluções propostas.

2. DISCUSSÃO

2.1. GPU x CPU

- CPU = tem poucos núcleos (4-64), onde cada núcleo é rápido; bom para tarefas complexas e variadas.
- GPU = milhares de núcleos, onde cada um é simples e especializado. Bom para tarefas repetidas.

2.1.1. Offloading

O programa sempre inicia na CPU. Quando quer usar a GPU, a CPU despacha (offloading) parte do trabalho para a GPU.

CPU prepara os dados -> envia para a GPU -> GPU processa -> devolve resultado para a CPU

2.1.2. `pragma omp target`

Essa diretiva que permite a realização do offloading.

Estrutura básica:

```
#pragma omp target map(to: a[0:N], b[0:N]) map(from: c[0:N])
```

```
{
#pragma omp parallel for
}
```

- Tudo o que estiver dentro das chaves após o target será executado na GPU;
- `map(to: a[0:N], b[0:N])`: enviar os dados da CPU para o GPU antes de começar, o array "a", do índice 0 até N;
- `map(from: c[0:N])`: ao finalizar, trazer os resultados para a CPU;
- `#pragma omp parallel for`: dentro do target, para que as milhares de threads da GPU possam dividir o trabalho.

Existe um custo para transferir a memória do CPU para o GPU e vice-versa, por isso às vezes pode parecer que a GPU seja mais lenta que a CPU em problemas pequenos.

2.1.3. `pragma omp loop`

Essa diretiva tem a mesma função do `pragma omp parallel for`, com a diferença na maneira que pede ao compilador para fazer essa otimização.

- `for`: a pessoa especifica como paralelizar;
- `loop`: o compilador decide a melhor forma de fazer essa paralelização.

A GPU é dividida em hierarquia interna de threads, e a diretiva com `loop` já foi criada pensando nisso, por isso consegue organizar as threads de forma mais eficiente para a GPU específica.

2.2. COMPILAR

Compilar no NPAD ``nvc -fast -mp=gpu vadd.c``

2.2.1. Submeter e ver o resultado

```
sbatch vadd.sh
squeue -u $USER
cat vadd-<id>.out
```

- `nvc`: compilador do Nvidia (em vez do `gcc` comum)
- `-fast`: otimização geral de performance

- mp=gpu: ativa o suporte openmp com offloading para GPU

2.3. RESOLUÇÃO EM CÓDIGO DA TAREFA

Segue no apêndice.

2.4. OUTPUT

Segue no apêndice.

3. EXPLICAÇÃO DA TAREFA

Verificação do tempo de processamento nas versões com CPU original, as versões GPU com target e for, e a versão com target e loop.

4. COMO RESOLVER

- 1º. Manter o loop CPU original e analisar o tempo;
- 2º. Versão GPU com #pragma omp target + #pragma omp parallel for, medir tempo;
- 3º. Versão GPU com #pragma omp target + #pragma omp loop, medir tempo.

5. CONCLUSÃO

Os resultados mostraram que, para esse problema específico de adição de vetores com 10 milhões de elementos, a CPU foi mais rápida que a GPU.

- CPU: 0.009s
- GPU parallel for: 0.567s
- GPU loop: 0.878s

5.1. CPU > GPU

O tempo de transferência de dados dominou o tempo de execução, isto é, o custo de comunicação superou o ganho do paralelismo da GPU.

5.2. GPU for > GPU loop

Na versão com loop, o compilador pode ter adicionado uma camada de organização desnecessária que custou tempo e por isso ficou com tempo maior que a versão com for.

6. APÊNDICE

6.1. RESOLUÇÃO

6.1.1. Versão pra compilar na CPU

```
#include <stdio.h>
#include <omp.h>
#define N 10000000
#define TOL 0.0000001
//
// This is a simple program to add two vectors
// and verify the results.
//
// History: Written by Tim Mattson, November 2017
//
int main()
{
    static float a[N], b[N], c[N], res[N];
    int err=0;

    double init_time, compute_time, test_time;
    init_time = -omp_get_wtime();

    // fill the arrays
    for (int i=0; i<N; i++){
        a[i] = (float)i;
        b[i] = 2.0*(float)i;
        c[i] = 0.0;
        res[i] = i + 2*i;
    }

    init_time += omp_get_wtime();
    compute_time = -omp_get_wtime();

    // add two vectors
    for (int i=0; i<N; i++){
        c[i] = a[i] + b[i];
    }

    compute_time += omp_get_wtime();
    test_time = -omp_get_wtime();

    // test results
    for(int i=0;i<N;i++){
        float val = c[i] - res[i];
        val = val*val;
        if(val>TOL) err++;
    }
}
```

```

test_time    += omp_get_wtime();

printf(" vectors added with %d errors\n",err);

printf("Init time:    %.3fs\n", init_time);
printf("Compute time: %.3fs\n", compute_time);
printf("Test time:    %.3fs\n", test_time);
printf("Total time:   %.3fs\n", init_time + compute_time + test_time);
return 0;
}

```

6.1.2. Versão GPU target + for

```

#include <stdio.h>
#include <omp.h>
#define N 10000000
#define TOL 0.0000001

int main()
{

    static float a[N], b[N], c[N], res[N]; // inicializar vetores
    // adicionei static pra não crashar
    int err=0; // taxa de erro 0

    double init_time, compute_time, test_time;
    init_time    = -omp_get_wtime();

    // fill the arrays
    for (int i=0; i<N; i++){
        a[i] = (float)i;
        b[i] = 2.0*(float)i;
        c[i] = 0.0;
        res[i] = i + 2*i;
    }

    init_time    += omp_get_wtime(); // iniciar tempo
    compute_time = -omp_get_wtime();

    // fazer pra versão GPU: target + for
    #pragma omp target map(to: a[0:N], b[0:N]) map(tofrom: c[0:N])
    {
        #pragma omp parallel for
        for (int i=0; i<N; i++){
            c[i] = a[i] + b[i];
        }
    }
}

```

```

compute_time += omp_get_wtime();
test_time     = -omp_get_wtime();

// test results
for(int i=0;i<N;i++){
    float val = c[i] - res[i];
    val = val*val;
    if(val>TOL) err++;
}

test_time     += omp_get_wtime();

printf(" vectors added with %d errors\n",err);

printf("Init time:      %.3fs\n", init_time);
printf("Compute time: %.3fs\n", compute_time);
printf("Test time:      %.3fs\n", test_time);
printf("Total time:     %.3fs\n", init_time + compute_time + test_time);
return 0;
}

```

6.1.3. Versão GPU target + loop

```

#include <stdio.h>
#include <omp.h>
#define N 10000000
#define TOL 0.0000001

int main()
{

    static float a[N], b[N], c[N], res[N]; // inicializar vetores
    // adicionei static pra não crashar
    int err=0; // taxa de erro 0

    double init_time, compute_time, test_time;
    init_time     = -omp_get_wtime();

    // fill the arrays
    for (int i=0; i<N; i++){
        a[i] = (float)i;
        b[i] = 2.0*(float)i;
        c[i] = 0.0;
    }
}

```

```

    res[i] = i + 2*i;
}

init_time    += omp_get_wtime(); // iniciar tempo
compute_time = -omp_get_wtime();

// fazer pra versão GPU: target + loop
#pragma omp target map(to: a[0:N], b[0:N]) map(tofrom: c[0:N])
{
    #pragma omp loop
    for (int i=0; i<N; i++){
        c[i] = a[i] + b[i];
    }
}

compute_time += omp_get_wtime();
test_time    = -omp_get_wtime();

// test results
for(int i=0; i<N; i++){
    float val = c[i] - res[i];
    val = val*val;
    if(val>TOL) err++;
}

test_time    += omp_get_wtime();

printf(" vectors added with %d errors\n",err);

printf("Init time:      %.3fs\n", init_time);
printf("Compute time:   %.3fs\n", compute_time);
printf("Test time:       %.3fs\n", test_time);
printf("Total time:     %.3fs\n", init_time + compute_time + test_time);
return 0;
}

```

6.2. OUTPUT

6.2.1. Versão CPU

```
[[cvnsantos@service0 tarefa19]$ ls
vaad_targetFor      vaad_targetFor.sh  vaad_targetLoop.c  vadd.c
vaad_targetFor.c    vaad_targetLoop    vaad_targetLoop.sh vadd_cpu
[[cvnsantos@service0 tarefa19]$ ./vadd_cpu
vectors added with 0 errors
Init time:      0.100s
Compute time:   0.009s
Test time:      0.007s
Total time:     0.116s
```

6.2.2. Versão GPU

```
[[cvnsantos@service0 tarefa19]$ cat vaad_targetFor-1855463.out
vectors added with 0 errors
Init time:      0.056s
Compute time:   0.567s
Test time:      0.007s
Total time:     0.631s
[[cvnsantos@service0 tarefa19]$ cat vaad_targetLoop-1855464.out
vectors added with 0 errors
Init time:      0.054s
Compute time:   0.878s
Test time:      0.007s
Total time:     0.939s
[[cvnsantos@service0 tarefa19]$
```